# UNIT 3

# Exercise 3 : Document storage and top-k retrieval

## Aim

To generate numerical embeddings for text and identify semantically similar sentences using embedding similarity.

## Algorithm

- Start the program.
- Load a pretrained sentence-embedding model.
- Define a collection of text documents.
- Convert the documents into numerical vectors.
- Convert the user's query into an embedding.
- Calculate similarity scores between the query and documents.
- Display the most similar document.
- Stop.

## Python Code and Output

```
# Document storage: IDs mapped to document text
document_store = {
    i: text for i, text in enumerate(documents)
}

# The FAISS index from Experiment 3 can be reused.
query = "How can documents help an AI answer questions?"
k = 3

query_vector = model.encode(
    [query], normalize_embeddings=True
).astype("float32")

scores, indices = index.search(query_vector, k)

print("Top-k retrieval results:")

for rank, (score, idx) in enumerate(
    zip(scores[0], indices[0]), start=1
):
    print(f"Rank {rank}, Document ID: {idx}")
    print("Content:", document_store[int(idx)])
    print("Score:", round(float(score), 4))
    print()
```

```
Top-k retrieval results:
Rank 1, Document ID: 5
Content: Retrieval augmented generation uses documents to answer questions.
Score: 0.5659
```

## Result

Text embeddings were generated successfully, and semantic similarity search identified the most relevant document.